第1章　装作自己是个小白——初识MySQL

1.1　MySQL 的客户端 / 服务器架构

以我们平时使用的微信为例，它其实是由客户端程序（可以简称为客户端）和服务器程序（可以简称为服务器）两部分组成的。微信客户端可能具有多种形式，比如手机 App、桌面端的软件或者网页版的微信。微信的每个客户端都有一个唯一的用户名，即微信号。另一方面，腾讯公司在它们的机房里运行着微信的服务器程序。我们平时在微信上的各种操作，其实就是使用微信客户端与微信服务器打交道。比如狗哥使用微信给猫爷发一条微信消息的过程大致如下所示。

1. 狗哥发出的微信消息被客户端进行包装，添加了发送者和接收者的信息，然后从客户端发送到微信服务器。
2. 微信服务器从收到的消息中获取发送者和接收者信息，并据此将消息发送到猫爷的微信客户端。然后，猫爷的微信客户端就会显示狗哥给他发的消息。

MySQL 的运行过程与之类似，即它的服务器程序直接与要存储的数据打交道，多个客户端程序可以连接到这个服务器程序，向服务器发送增删查改的请求，然后服务器程序根据这些请求，对存储的数据进行相应处理。与微信一样，MySQL 的每一个客户端都需要使用用户名和密码才能登录服务器，而且只有在登录之后才能向服务器发送某些请求来操作数据。MySQL 的日常使用场景是下面这样的。

1. 启动 MySQL 服务器程序。
2. 启动 MySQL 客户端程序，并连接到服务器程序。
3. 在客户端程序中输入命令语句，并将其作为请求发送给服务器程序。服务器程序在收到这些请求后，根据请求的内容来操作具体的数据，并将结果返回给客户端。

众所周知，现在计算机的功能都很强大，一台计算机上可以同时运行多个程序，比如微信、QQ、英雄联盟游戏、文本编辑器等。计算机上运行的每一个程序也称为一个进程。运行过程中的 MySQL 服务器程序和客户端程序在本质上来说都算是计算机中的进程，其中代表 MySQL 服务器程序的进程称为 MySQL 数据库实例（instance）。

1.2　MySQL 的安装

在安装 MySQL 时，无论是通过下载源代码的方式自行编译安装，还是直接使用官方提供的安装包进行安装，MySQL 的服务器程序和客户端程序都会安装到机器上。无论采用上述哪种安装方式，一定要记住 MySQL 安装在哪里了。换句话说，一定一定一定要记住 MySQL 的安装目录。

小贴士

MySQL 的大部分安装包都包含了服务器程序和客户端程序，不过在 Linux 环境下可以使用不同的 RPM 包分别安装服务器程序和客户端程序。具体安装细节可以参阅 MySQL 文档。

另外，MySQL 可以运行在各种类型的操作系统上，本书后文会讨论 MySQL 在类 UNIX 操作系统和 Windows 操作系统上的一些使用差别。为了方便大家理解，我在 macOS 操作系统和 Windows 操作系统上都安装了 MySQL，它们的安装目录分别如下。

- macOS 操作系统上的安装目录：/usr/local/mysql/。
- Windows 操作系统上的安装目录：C:\Program Files\MySQL\MySQL Server 5.7\。

下面会以这两个安装目录为例来进一步引出更多的概念。大家一定要注意，上面这两个安装目录是在我的运行不同操作系统的机器上的安装目录，大家在后文的示例中一定要将安装目录替换为自己机器上的安装目录。

小贴士

类 UNIX 操作系统非常多，比如 FreeBSD、Linux、macOS、Solaris 等都属于 UNIX 操作系统的范畴。这里使用 macOS 操作系统代表类 UNIX 操作系统来运行 MySQL。

1.2.1 bin 目录下的可执行文件

在 MySQL 的安装目录下有一个特别重要的 bin 目录，这个目录存放着许多可执行文件。以 macOS 系统为例，这个 bin 目录的绝对路径（在我的机器上）就是 /usr/local/mysql/bin。

下面我们列出 macOS 系统中这个 bin 目录下的部分可执行文件，如下所示。需要说明的是，该目录下的文件太多，这里仅列出了部分。

```
.
├── mysql
├── mysql.server -> ../support-files/mysql.server
├── mysqladmin
├── mysqlbinlog
├── mysqlcheck
├── mysqld
├── mysqld_multi
├── mysqld_safe
├── mysqldump
├── mysqlimport
├── mysqlpump
... （省略其他文件）
0 directories, 40 files
```

Windows 中的可执行文件与 macOS 中的类似，不过都是以 .exe 为扩展名（不同的操作系统中，bin 目录下包含的可执行文件并不完全相同）。这些可执行文件中，有的是服务器程序，有的是客户端程序。后面会详细介绍一些比较重要的可执行文件，这里先看一下这些文件的执行方式。

在具有图形用户界面的操作系统中，可以通过鼠标点击的方式打开并运行某个可执行文件。但是我们现在要关注的是，如何在命令行解释器下运行这些可执行文件。

小贴士

所谓命令行解释器，指的就是类 UNIX 系统中的 Shell 或者 Windows 系统中的 cmd.exe。因为命令行解释器的界面通常是一个黑框框，所以在后文中我们可能会直接把命令行解释器称作黑框框。

下面以 macOS 系统为例来看看如何执行这些可执行文件（Windows 中的操作与之类似，这里不再赘述）。

- 使用可执行文件的相对 / 绝对路径来执行。

假设命令行解释器中的当前工作目录是 MySQL 的安装目录，即 /usr/local/mysql，要想执行 bin 目录下的 mysqld 可执行文件，可以使用相对路径，如下所示：

```
./bin/mysqld
```

也可以通过直接输入 mysqld 的绝对路径的方式来执行，如下所示：

```
/usr/local/mysql/bin/mysqld
```

- 将 bin 目录的绝对路径加入到环境变量 PATH 中。

大家应该发现，若每次执行一个文件都需要输入一长串路径名，这未免太麻烦了。针对这种情况，可以把 bin 目录所对应的绝对路径添加到环境变量 PATH 中。环境变量 PATH 是一系列路径的集合，各个路径之间使用冒号（:）隔离开。

比如，在我的机器上，环境变量 PATH 的值为 /usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin。这个值表明，在我输入某个命令时，系统会在 /usr/local/bin、/usr/bin、/bin、/usr/sbin 和 /sbin 目录下按照顺序依次寻找输入的这个命令。如果寻找成功，则执行该命令。

也可以修改这个环境变量 PATH，把 MySQL 安装目录下的 bin 目录的绝对路径添加到 PATH 中。修改后的环境变量 PATH 的值为 /usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:/usr/local/mysql/bin。这样一来，无论命令行解释器的当前工作目录是啥，都可以直接输入可执行文件的名字来启动，比如下面这样：

```
mysqld
```

可见，这样一来方便多了！

小贴士

有关环境变量的概念以及在系统中修改环境变量的方法不属于本书讲解范围，大家可以自行查询相关资料进行更深入的学习。

1.3 启动 MySQL 服务器程序

1.3.1 在类 UNIX 系统中启动服务器程序

在类 UNIX 系统中，用来启动 MySQL 服务器程序的可执行文件有很多，且大部分都位于 MySQL 安装目录的 bin 目录下。下面我们一探究竟。

1. mysqld

mysqld 可执行文件就表示 MySQL 服务器程序，运行这个可执行文件就可以直接启动一个 MySQL 服务器进程。但这个可执行文件并不常用，我们继续看其他的启动命令。

2. mysqld_safe

mysqld_safe 是一个启动脚本，它会间接调用 mysqld 并持续监控服务器的运行状态。当服务器进程出现错误时，它还可以帮助重启服务器程序。另外，使用 mysqld_safe 启动 MySQL 服务器程序时，它会将服务器程序的出错信息和其他诊断信息输出到错误日志，以方便后期查找发生错误的原因。

小贴士

出错日志默认写到一个以 .err 为扩展名的文件中，该文件位于 MySQL 的数据目录中。后续章节会介绍 MySQL 的这个数据目录。

3. mysql.server

mysql.server 也是一个启动脚本，它会间接地调用 mysqld_safe。在执行 mysql.server 时，在后面添加 start 参数就可以启动服务器程序了，如下所示：

```
mysql.server start
```

需要注意的是，mysql.server 文件其实是一个链接文件，它对应的实际文件是 ../support-files/mysql.server。

小贴士

通过源码安装 MySQL，或者使用一个没有自动安装 mysql.server 脚本的安装包来安装 MySQL 时，需要手动安装这个 mysql.server 脚本。具体安装方式可以参阅相关文档。

还可以使用 mysql.server 来关闭正在运行的服务器程序，此时只需把 start 参数换成 stop 即可，如下所示：

```
mysql.server stop
```

4. mysqld_multi

其实我们在一台计算机上也可以运行多个服务器实例，也就是运行多个 MySQL 服务器进程。mysqld_multi 可执行文件可以启动或停止多个服务器进程，也能报告它们的运行状态。这个命令的使用比较复杂，本书主要是为了讲清楚单个 MySQL 服务器的运行过程，不会对启动多个服务器程序进行过多唠叨。

小贴士

mysqld_safe、mysql.server 和 mysqld_multi 本质上是一个 Shell 脚本，大家可以直接把它们当作文本打开并进行浏览（前提是能看懂 Shell 脚本）。

1.3.2 在 Windows 系统中启动服务器程序

尽管在 Windows 中没有提供像 UNIX 中的那么多启动脚本，但是它也提供了两种启动方

法，分别是手动启动和以服务的形式启动。

1. 手动启动

在 Windows 系统中安装完 MySQL 之后，MySQL 安装目录的 bin 目录下也会存在一个 mysqld 可执行文件。在命令行解释器中输入 mysqld，或者直接在 bin 目录下双击该文件，就可以启动 MySQL 服务器程序了。

小贴士

如果没有启动成功，可以尝试使用 mysqld --console 命令来启动服务器程序，这样可以把启动过程中生成的错误信息在黑框框中显示出来，以方便我们定位错误。

2. 以服务的方式启动

如果我们需要在计算机上长时间运行某个程序，并且无论是谁在使用这台计算机，这个程序的运行都不受影响，我们就可以把它注册为一个 Windows 服务，由操作系统来帮我们管理。把某个程序注册为 Windows 服务的方式挺简单，具体如下：

```
"完整的可执行文件路径" --install [-manual] [服务名]
```

如果我们添加了 -manual 选项，就表示在 Windows 系统启动的时候不自动启动该服务，否则会自动启动。服务名也可以省略，默认的服务名就是 MySQL。比如我的 Windows 计算机上 mysqld 的完整路径是：

```
C:\Program Files\MySQL\MySQL Server 5.7\bin\mysqld
```

所以如果我们想把它注册为 Windows 服务，可以在黑框框中这么写：

```
"C:\Program Files\MySQL\MySQL Server 5.7\bin\mysqld" --install
```

在把 mysqld 注册为 Windows 服务之后，就可以通过下面这个命令来启动 MySQL 服务器程序了：

```
net start MySQL
```

当然，如果你喜欢图形界面，可以通过 Windows 的服务管理器并用鼠标点击的方式来启动和停止服务。

关闭这个服务也非常简单，只要把上面的 start 换成 stop 就行了，就像下面这样：

```
net stop MySQL
```

1.4 启动 MySQL 客户端程序

在成功启动 MySQL 服务器程序后，就可以启动客户端程序来连接到这个服务器了。bin 目录下有许多客户端程序，比方说 mysqladmin、mysqldump、mysqlcheck 等。这里要重点关注的是可执行文件 mysql（指的是一个名称为 mysql 的可执行文件）。通过这个可执行文件，我们可以与服务器程序交互，也就是发送请求并接收服务器的处理结果。启动这个可执行文件时，一般需要一些参数，格式如下：

```
mysql -h主机名  -u用户名 -p密码
```

各个参数的意义如表 1-1 所示。

表 1-1 启动客户端程序时需要的参数及其含义

参数名	含义
-h	表示服务器进程所在计算机的域名或者 IP 地址。如果服务器进程就运行在本机的话，可以省略这个参数，或者填写 localhost 或 127.0.0.1；也可以写成“--host= 主机名”的形式
-u	表示用户名；也可以写成“--user= 用户名”的形式
-p	表示密码；也可以写成“--password= 密码”的形式

小贴士

像 h、u、p 这种名称只有一个英文字母的参数称为短形式的参数，使用时前面需要加单短划线；像 host、user、password 这种有多个字母组成的参数称为长形式的参数，使用时前面需要加双短划线。后文会详细讨论这些参数的使用方式，现在少安毋躁。

比如我们在黑框框中执行下面这个语句（用户名和密码按实际情况填写），就可以启动 MySQL 客户端，并且连接到服务器了。

```
mysql -hlocalhost -uroot -p123456
```

我们看一下连接成功后的界面：

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 5.7.22-debug-log Source distribution

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

最后一行的 mysql> 是一个客户端的提示符，之后客户端发送给服务器的命令都需要写在这个提示符后边。

如果我们想断开客户端与服务器的连接并且关闭客户端的话，可以在 mysql> 提示符后输入下面任意一个命令：

- quit
- exit
- \q

比如输入 quit 试试：

```
mysql> quit
Bye
```

输出 Bye 说明客户端程序已经关掉了。大家一定要注意，这是关闭客户端程序的方式，而不是关闭服务器程序的方式。怎么关闭服务器程序已经在上一节唠叨过了。

如果愿意，可以多打开几个黑框框，并在每个黑框框都运行命令 mysql -hlocalhost -uroot -p123456，这样我们就启动了多个客户端程序，且每个客户端程序都是互不影响的。如果你有多台计算机，也可以试着把它们用局域网连起来，在一台计算机上启动 MySQL 服务器程序，在另一台计算机上执行 mysql 命令时使用 IP 地址作为主机名来连接到服务器。

1.4.1 连接注意事项

- 最好不要在一行命令中输入密码。

  在一些系统中，我们直接在黑框框中输入的密码可能会被同一台机器上的其他用户通过诸如 ps 之类的命令看到。也就是说这种方式并不安全，与你当着别人的面输入银行卡密码没啥区别。我们在执行 mysql 命令连接服务器的时候可以不显式地写出密码，就像下面这样：

  ```
  mysql -hlocalhost -uroot -p
  ```

  按回车键之后才会提示输入密码：

  ```
  Enter password:
  ```

  不过这次你输入的密码不会被显示出来，心怀不轨的人也就看不到了。输入完成后按回车键就成功连接到了服务器。

- 如果非要在一行命令中显式地输入密码，那么 -p 和密码值之间不能有空白字符（其他参数名和参数值之间可以有空白字符），就像下面这样：

  ```
  mysql -h localhost -u root -p123456
  ```

  如果在 -p 和密码值之间加上了空白字符就是错误的（下边的命令会导致服务器把 123456 当作数据库名称对待），比如这样：

  ```
  mysql -h localhost -u root -p 123456
  ```

- mysql 的各个参数的顺序没有硬性规定，也就是说我们也可以这么写：

  ```
  mysql -p  -u root -h localhost
  ```

- 如果你的服务器和客户端安装在同一台机器上，-h 参数可以省略，就像下面这样：

  ```
  mysql -u root -p
  ```

- 如果你使用的是类 UNIX 系统，在省略 -u 参数后，会把登录操作系统的用户名当作 MySQL 的用户名去处理。

  比如我用来登录操作系统的用户名是 xiaohaizi4919，那么下面这两条命令在我的机器上是等价的：

  ```
  mysql -u xiaohaizi4919 -p
  mysql -p
  ```

  对于 Windows 系统来说，默认的用户名是 ODBC，可以通过设置环境变量 USER 来添加一个默认用户名。

1.5 客户端与服务器连接的过程

我们现在已经知道如何启动 MySQL 的服务器程序，以及如何启动客户端程序来连接到这个服务器程序。运行中的服务器程序和客户端程序本质上都是计算机上的一个进程，所以客户端进程向服务器进程发送请求并得到响应的过程本质上是一个进程间通信的过程！ MySQL 支持下面几种客户端进程和服务器进程的通信方式。

1.5.1 TCP/IP

在真实环境中，数据库服务器进程和客户端进程可能运行在不同的主机中，它们之间必须通过网络进行通信。MySQL 采用 TCP 作为服务器和客户端之间的网络通信协议。在网络环境下，每台计算机都有一个唯一的 IP 地址，如果某个进程需要采用 TCP 协议进行网络通信，就可以向操作系统申请一个端口号。端口号是一个整数值，它的取值范围是 0 ～ 65535。这样，网络中的其他进程就可以通过 IP 地址＋端口号的方式与这个进程建立连接，这样进程之间就可以通过网络进行通信了。

MySQL 服务器在启动时会默认申请 3306 端口号，之后就在这个端口号上等待客户端进程进行连接。用书面一点的话来说，MySQL 服务器会默认监听 3306 端口。

小贴士

TCP/IP 是现在通用的一种网络体系结构，其中 TCP 和 IP 是两个非常重要的网络协议。如果你不知道协议是什么，或者不知道网络是什么，赶紧找本计算机网络的书瞅瞅吧。如果看得很吃力的话可以等我喔。

如果 3306 端口号已经被别的进程占用，或者我们单纯地想自定义该服务器进程监听的端口号，就可以在启动服务器程序的命令行中添加 -P 参数来明确指定端口号，比如这样：

```
mysqld -P3307
```

这样 MySQL 服务器在启动时就会去监听指定的端口号 3307。

如果客户端进程想要使用 TCP/IP 网络与服务器进程进行通信，那么我们在使用 mysql 命令启动客户端程序时，在 -h 参数后必须跟随 IP 地址来作为需要连接的服务器进程所在主机的主机名。如果客户端进程和服务器进程位于同一台计算机中，则可以使用 127.0.0.1 来代表本机的 IP 地址。另外，如果服务器进程监听的端口号不是默认的 3306，我们也可以在使用 mysql 命令启动客户端程序时使用 -P 参数（注意是大写的 P，小写的 p 是用来指定密码的）指定需要连接的端口号。比如我们现在已经在本机启动了服务器，监听的端口号为 3307，我们在启动客户端程序时可以这样写：

```
mysql -h127.0.0.1 -uroot -P3307 -p
```

1.5.2 命名管道和共享内存

如果你是一位 Windows 用户，那么可以考虑在客户端进程和服务器进程之间使用命名管道或共享内存进行通信。不过在使用这些通信方式的时候，需要在启动服务器程序和客户端程序时添加一些参数。

- 使用命名管道进行进程间通信：需要在启动服务器程序的命令中加上 --enable-named-pipe 参数，然后在启动客户端程序的命令中加上 --pipe 或者 --protocol=pipe 参数。
- 使用共享内存进行进程间通信：需要在启动服务器程序的命令中加上 --shared-memory 参数。在成功启动服务器后，共享内存便成为本地客户端程序的默认连接方式。我们也可以在启动客户端程序的命令中加上 --protocol=memory 参数来显式指定使用共享内存进行通信。

需要注意的是，使用共享内存进行通信的服务器进程和客户端进程必须位于同一台 Windows 主机中。

小贴士

命名管道和共享内存是 Windows 操作系统中的两种进程间通信方式，如果没听过也不用纠结，这并不妨碍我们学习 MySQL 的知识。

1.5.3 UNIX 域套接字

如果服务器进程和客户端进程都运行在操作系统为类 UNIX 的同一台机器上，则可以使用 UNIX 域套接字进行进程间通信。如果在启动客户端程序时没有指定主机名，或者指定的主机名为 localhost，又或者指定了 --protocol=socket 的启动参数，那么服务器程序和客户端程序之间就可以通过 UNIX 域套接字进行通信了。

MySQL 服务器程序默认监听的 UNIX 域套接字文件名称为 /tmp/mysql.sock，客户端程序也默认连接到这个 UNIX 域套接字文件名称。如果想改变这个默认的名称，可以在启动服务器程序时指定 socket 参数，就像下面这样：

```
mysqld --socket=/tmp/a.txt
```

这样服务器在启动后便会监听 /tmp/a.txt。在服务器改变了默认的 UNIX 域套接字文件名称后，如果客户端程序想通过 UNIX 域套接字进行通信，也需要显式地指定连接的 UNIX 域套接字文件名称，就像下面这样：

```
mysql -hlocalhost -uroot --socket=/tmp/a.txt -p
```

小贴士

如果大家不了解啥是 UNIX 域套接字也不用深究了，这里只是为了内容的完整性才写的。大家只要知道有这么一种进程间通信的方式就好了，对我们理解后续的内容并没有影响。

1.6 服务器处理客户端请求

其实，无论客户端进程和服务器进程采用哪种方式进行通信，最后实现的效果都是客户端进程向服务器进程发送一段文本（MySQL 语句），服务器进程处理后再向客户端进程返回一段文本（处理结果）。那么，服务器进程对客户端进程发送的请求做了什么处理，才能产生最

后的处理结果呢？客户端可以向服务器发送增删改查等各类请求，这里以比较复杂的查询请求为例来展示一下大致的过程，如图 1-1 所示。

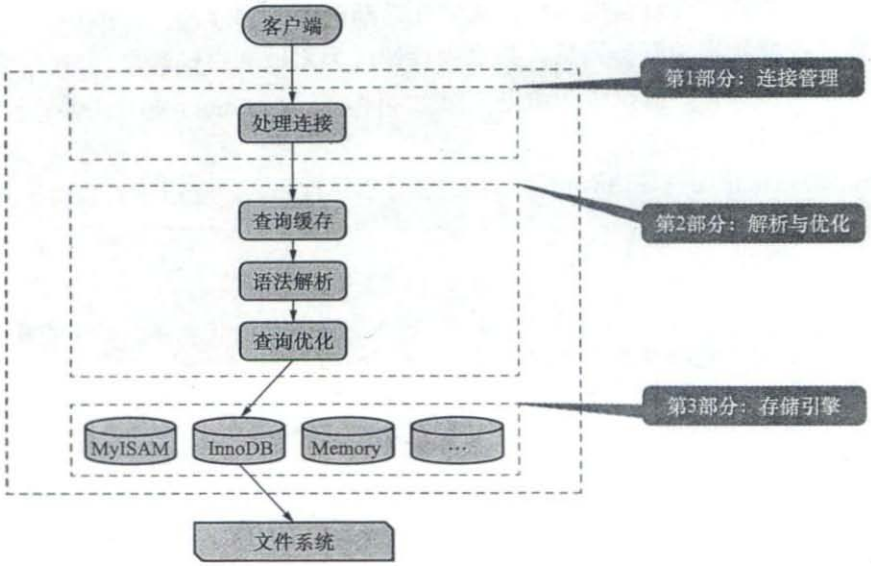

图 1-1 查询请求执行过程

从图 1-1 中可以看出，服务器程序在处理来自客户端的查询请求时，大致需要分为 3 部分：连接管理、解析与优化、存储引擎。下边来详细看一下这 3 部分都做了些什么。

1.6.1 连接管理

客户端进程可以采用前面介绍的 TCP/IP、命名管道或共享内存、UNIX 域套接字等几种方式与服务器进程建立连接。每当有一个客户端进程连接到服务器进程时，服务器进程都会创建一个线程专门处理与这个客户端的交互；当该客户端退出时会与服务器断开连接，服务器并不会立即把与该客户端交互的线程销毁，而是把它缓存起来，在另一个新的客户端再进行连接时，把这个缓存的线程分配给该新客户端。这样就不用频繁地创建和销毁线程，从而节省了开销。从这一点大家也能看出，MySQL 服务器会为每一个连接进来的客户端分配一个线程，但是线程分配得太多会严重影响系统性能，所以我们也需要限制可以同时连接到服务器的客户端数量，至于怎么限制我们后边再说。

在客户端程序发起连接时，需要携带主机信息、用户名、密码等信息，服务器程序会对客户端程序提供的这些信息进行认证。如果认证失败，服务器程序会拒绝连接。另外，如果客户端程序和服务器程序不运行在一台计算机上，我们还可以通过采用传输层安全性（Transport Layer Security，TLS）协议对连接进行加密，从而保证数据传输的安全性。

当连接建立后，与该客户端关联的服务器线程会一直等待客户端发送过来的请求。MySQL 服务器接收到的请求只是一个文本消息，该文本消息还要经过各种处理。预知后事如何，请继续往下看。

1.6.2 解析与优化

到现在为止，MySQL 服务器已经获得了文本形式的请求，接着还要经过“九九八十一难”

的处理，其中几个比较重要的部分分别是查询缓存、语法解析和查询优化。下面我们详细来看。

1. 查询缓存

如果我问你 9 + 8 × 16 − 3 × 2 × 17 的值是多少，你可能会用计算器去算一下，或者再厉害一点直接用心算，最终得到了结果 35。如果我再问你一遍 9 + 8 × 16 − 3 × 2 × 17 的值是多少，你还用再傻呵呵地算一遍么？我们刚刚已经算过了，直接说答案就好了。

MySQL 服务器程序处理查询请求的过程也是这样，会把刚刚处理过的查询请求和结果缓存起来。如果下一次有同样的请求过来，直接从缓存中查找结果就好了，就不用再去底层的表中查找了。这个查询缓存可以在不同的客户端之间共享，也就是说，如果客户端 A 刚刚发送了一个查询请求，而客户端 B 之后发送了同样的查询请求，那么客户端 B 的这次查询就可以直接使用查询缓存中的数据了。

当然，MySQL 服务器并没有人那么聪明，如果两个查询请求有任何字符上的不同（例如，空格、注释、大小写），都会导致缓存不会命中。另外，如果查询请求中包含某些系统函数、用户自定义变量和函数、系统表，如 mysql、information_schema、performance_schema 数据库中的表，则这个请求就不会被缓存。以某些系统函数为例，同一个函数的两次调用可能会产生不一样的结果。比如函数 NOW，每次调用时都会产生最新的当前时间。如果在两个查询请求中调用了这个函数，即使查询请求的文本信息都一样，那么不同时间的两次查询也应该得到不同的结果。如果在第一次查询时就缓存了结果，在第二次查询时直接使用第一次查询的结果就是错误的！

不过既然是缓存，那就有缓存失效的时候。MySQL 的缓存系统会监测涉及的每张表，只要该表的结构或者数据被修改，比如对该表使用了 INSERT、UPDATE、DELETE、TRUNCATE TABLE、ALTER TABLE、DROP TABLE 或 DROP DATABASE 语句，则与该表有关的所有查询缓存都将变为无效并从查询缓存中删除！

小贴士

虽然查询缓存有时可以提升系统性能，但也不得不因维护这块缓存而造成一些开销。比如每次都要去查询缓存中检索，查询请求处理完后需要更新查询缓存，需要维护该查询缓存对应的内存区域等。从 MySQL 5.7.20 开始，不推荐使用查询缓存，在 MySQL 8.0 中直接将其删除。

2. 语法解析

如果查询缓存没有命中，接下来就需要进入正式的查询阶段了。因为客户端程序发送过来的请求只是一段文本，所以 MySQL 服务器程序首先要对这段文本进行分析，判断请求的语法是否正确，然后从文本中将要查询的表、各种查询条件都提取出来放到 MySQL 服务器内部使用的一些数据结构上。

小贴士

从本质上来说，这个从指定的文本中提取出需要的信息算是一个编译过程，涉及词法解析、语法分析、语义分析等阶段。这些问题不属于我们讨论的范畴，大家只要了解在处理请求的过程中需要这个步骤就好了。

3. 查询优化

在语法解析之后，服务器程序获得到了需要的信息，比如要查询的表和列是哪些、搜索

条件是什么等。但光有这些是不够的，因为我们写的 MySQL 语句执行起来效率可能并不是很高，MySQL 的优化程序会对我们的语句做一些优化，如外连接转换为内连接、表达式简化、子查询转为连接等一堆东西。优化的结果就是生成一个执行计划，这个执行计划表明了应该使用哪些索引执行查询，以及表之间的连接顺序是啥样，等等。我们可以使用 EXPLAIN 语句来查看某个语句的执行计划。关于查询优化的详细内容我们后边会仔细唠叨，现在只需要知道在 MySQL 服务器程序处理请求的过程中有这么一个步骤就好了。

1.6.3 存储引擎

到服务器程序完成了查询优化为止，还没有真正地去访问真实的表中数据（在查询优化期间可能访问表中少量数据，在讨论查询优化的章节中我们会详细唠叨）。MySQL 服务器把数据的存储和提取操作都封装到了一个名为存储引擎的模块中。我们知道，表是由一行一行的记录组成的，但这只是一个逻辑上的概念。在物理上如何表示记录，怎么从表中读取数据，以及怎么把数据写入具体的物理存储器上，都是存储引擎负责的事情。为了实现不同的功能，MySQL 提供了各式各样的存储引擎，不同存储引擎管理的表可能有不同的存储结构，采用的存取算法也可能不同。

小贴士

为什么叫引擎呢？可能这个名字更拉风吧。其实这个存储引擎以前叫作表处理器，后来可能人们觉得太土，就改成了存储引擎。它的功能就是接收上层传下来的指令，然后对表中的数据进行读取或写入操作。

为了方便管理，人们把 MySQL 服务器处理请求的过程简单地划分为 server 层和存储引擎层。连接管理、查询缓存、语法解析、查询优化这些并不涉及真实数据存取的功能划分为 server 层的功能，存取真实数据的功能划分为存储引擎层的功能。各种不同的存储引擎为 server 层提供统一的调用接口，其中包含了几十个不同用途的底层函数，比如“读取索引第一条记录”“读取索引下一条记录”“插入记录”等。

所以在 server 层完成了查询优化后，只需按照生成的执行计划调用底层存储引擎提供的接口获取到数据后返回给客户端就好了。不过需要注意的一点是，server 层和存储引擎层交互时，一般是以记录为单位的。以 SELECT 语句为例，server 层根据执行计划先向存储引擎层取一条记录，然后判断是否符合 WHERE 条件；如果符合，就发送给客户端，否则跳过该记录，然后继续向存储引擎索要下一条记录；依此类推。

小贴士

server 层在判断某条记录符合要求之后，其实是先将其发送到一个缓冲区，待到该缓冲区满了，才向客户端发送真正的记录。该缓冲区大小由系统变量 net_buffer_length 控制，当然，你现在可能不知道啥是系统变量，不过下一章就会知道了。

1.7 常用存储引擎

MySQL 支持多种存储引擎，先来看看表 1-2 中列出的部分存储引擎。

表 1-2 MySQL 支持的存储引擎

存储引擎	描述
ARCHIVE	用于数据存档（记录插入后不能再修改）
BLACKHOLE	丢弃写操作，读操作会返回空内容
CSV	在存储数据时，以逗号分隔各个数据项
FEDERATED	用来访问远程表
InnoDB	支持事务、行级锁、外键
MEMORY	数据只存储在内存，不存储在磁盘；多用于临时表
MERGE	用来管理多个 MyISAM 表构成的表集合
MyISAM	主要的非事务处理存储引擎
NDB	MySQL 集群专用存储引擎

这么多存储引擎，看着都眼花了，我们怎么挑啊。其实大家多虑了，我们最常用的就是 InnoDB 和 MyISAM，偶尔还会提一下 MEMORY。其中 InnoDB 是 MySQL 默认的存储引擎，我们之后会详细唠叨这个存储引擎的各种功能，现在先看一下部分存储引擎对于某些功能的支持情况，如表 1-3 所示。

表 1-3 存储引擎对于某些功能的支持情况

功能	MyISAM	MEMORY	InnoDB	ARCHIVE	NDB
B-tree indexes	是	是	是	否	否
Backup/point-in-time recovery	是	是	是	是	是
Cluster database support	否	否	否	否	是
Clustered indexes	否	否	是	否	否
Compressed data	是	否	是	是	否
Data caches	否	N/A	是	否	是
Encrypted data	是	是	是	是	是
Foreign key support	否	否	是	否	是
Full-text search indexes	是	否	是	否	否
Geospatial data type support	是	否	是	是	是
Geospatial indexing support	是	否	是	否	否
Hash indexes	否	是	否	否	是
Index caches	是	N/A	是	否	是
Locking granularity	表	表	行	行	行
MVCC	否	否	是	否	否
Query cache support	是	是	是	是	是
Replication support	是	有限支持	是	是	是
Storage limits	256TB	RAM	64TB	无存储限制	384EB
T-tree indexes	否	否	否	否	是
Transactions	否	否	是	否	是
Update statistics for data dictionary	是	是	是	是	是

表 1-3 密密麻麻列了这么多，看得让人头皮发麻，目的就是想告诉你：这玩意儿很复杂（其实表 1-3 是我从 MySQL 文档中直接复制过来的）。其实这些东西大家没必要立即记住，这里主要是想让大家明白不同的存储引擎支持不同的功能。有些重要的功能我们会在后面的唠叨中慢慢让大家理解。

小贴士

InnoDB 从 MySQL 5.5.5 版本开始作为 MySQL 的默认存储引擎，之前版本的默认存储引擎为 MyISAM。

1.8 关于存储引擎的一些操作

1.8.1 查看当前服务器程序支持的存储引擎

我们可以用下面这个命令来查看当前服务器程序支持的存储引擎：

```
SHOW ENGINES;
```

来看一下调用效果：

```
mysql> SHOW ENGINES;
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
| Engine             | Support | Comment                                                        | Transactions | XA   | Savepoints |
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
| InnoDB             | DEFAULT | Supports transactions, row-level locking, and foreign keys     | YES          | YES  | YES        |
| MRG_MYISAM         | YES     | Collection of identical MyISAM tables                          | NO           | NO   | NO         |
| MEMORY             | YES     | Hash based, stored in memory, useful for temporary tables      | NO           | NO   | NO         |
| BLACKHOLE          | YES     | /dev/null storage engine (anything you write to it disappears) | NO           | NO   | NO         |
| MyISAM             | YES     | MyISAM storage engine                                          | NO           | NO   | NO         |
| CSV                | YES     | CSV storage engine                                             | NO           | NO   | NO         |
| ARCHIVE            | YES     | Archive storage engine                                         | NO           | NO   | NO         |
| PERFORMANCE_SCHEMA | YES     | Performance Schema                                             | NO           | NO   | NO         |
| FEDERATED          | NO      | Federated MySQL storage engine                                 | NULL         | NULL | NULL       |
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
9 rows in set (0.00 sec)
```

其中，Support 列表示该存储引擎是否可用，DEFAULT 值代表当前服务器程序的默认存储引擎；Comment 列是对存储引擎的一个描述；Transactions 列代表该存储引擎是否支持事务处理；XA 列代表该存储引擎是否支持分布式事务；Savepoints 列代表该存储引擎是否支持事务的部分回滚。

小贴士

好吧，也许你并不知道什么是事务，更别提分布式事务了。关于事务的详细情况后续章节会非常详细地唠叨，少安毋躁。

1.8.2 设置表的存储引擎

我们前边说过，存储引擎是负责对表中的数据进行读取和写入工作的，我们可以为不同的表设置不同的存储引擎。也就是说，不同的表可以有不同的物理存储结构、不同的读取和写入方式。

1. 创建表时指定存储引擎

如果我们在创建表的语句中没有指定表的存储引擎，那就会使用默认的存储引擎 InnoDB

（当然，这个默认的存储引擎也是可以修改的，下一章中再说怎么改）。如果我们想显式地指定表的存储引擎，可以这么写：

```
CREATE TABLE 表名(
    建表语句;
) ENGINE = 存储引擎名称;
```

比如我们想创建一个存储引擎为 MyISAM 的表，可以这么写：

```
mysql> CREATE TABLE engine_demo_table(
    ->     i int
    -> ) ENGINE = MyISAM;
Query OK, 0 rows affected (0.02 sec)
```

2. 修改表的存储引擎

如果表已经建好了，我们也可以使用下面这个语句来修改表的存储引擎：

```
ALTER TABLE 表名 ENGINE = 存储引擎名称;
```

比如，我们修改 engine_demo_table 表的存储引擎：

```
mysql> ALTER TABLE engine_demo_table ENGINE = InnoDB;
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

这时我们再查看一下 engine_demo_table 的表结构：

```
mysql> SHOW CREATE TABLE engine_demo_table\G
*************************** 1. row ***************************
       Table: engine_demo_table
Create Table: CREATE TABLE 'engine_demo_table' (
  'i' int(11) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8
1 row in set (0.01 sec)
```

可以看到该表的存储引擎已经改为 InnoDB 了。

1.9 总结

MySQL 采用客户端 / 服务器架构，用户通过客户端程序发送增删改查请求，服务器程序收到请求后处理，并且把处理结果返回给客户端。

MySQL 安装目录的 bin 目录下存放了许多可执行文件，其中有一些是服务器程序（比如 mysqld、mysqld_safe），有一些是客户端程序（比如 mysql、mysqladmin）。

在类 UNIX 系统上启动服务器程序的方式有下面这些：

- mysqld；
- mysqld_safe；
- mysql.server；
- mysqld_multi。

在 Windows 系统上启动服务器程序的方式有下面这些：

- mysqld；
- 将 mysqld 注册为 Windows 服务。

启动客户端程序时常用的语法如下：

```
mysql -h主机名  -u用户名 -p密码
```

客户端进程和服务器进程在通信时采用下面几种方式：

- TCP/IP；
- 命名管道或共享内存；
- UNIX 域套接字。

以查询请求为例，服务器程序在处理客户端发送过来的请求时，大致分为以下几个部分。

- 连接管理：主要负责连接的建立与信息的认证。
- 解析与优化：主要进行查询缓存、语法解析、查询优化。
- 存储引擎：主要负责读取和写入底层表中的数据。

MySQL 支持的存储引擎有好多种，它们的功能各有侧重，我们常用的就是 InnoDB 和 MyISAM，其中 InnoDB 是服务器程序的默认存储引擎。

存储引擎的一些常用用法如下所示：

- 查看当前服务器程序支持的存储引擎：

```
SHOW ENGINES;
```

- 创建表时指定表的存储引擎：

```
CREATE TABLE 表名(
    建表语句;
) ENGINE = 存储引擎名称;
```

- 修改表的存储引擎：

```
ALTER TABLE 表名 ENGINE = 存储引擎名称;
```